In [1]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib as pt
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
```

In [2]:
```
df=pd.read_csv("insurance.csv")
df
```

Out[2]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
...	...	...	...	...	...	...	...
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

1338 rows × 7 columns

In [3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       1338 non-null   int64  
 1   sex       1338 non-null   object 
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64  
 4   smoker    1338 non-null   object 
 5   region    1338 non-null   object 
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

In [4]:
```
df.describe()
```

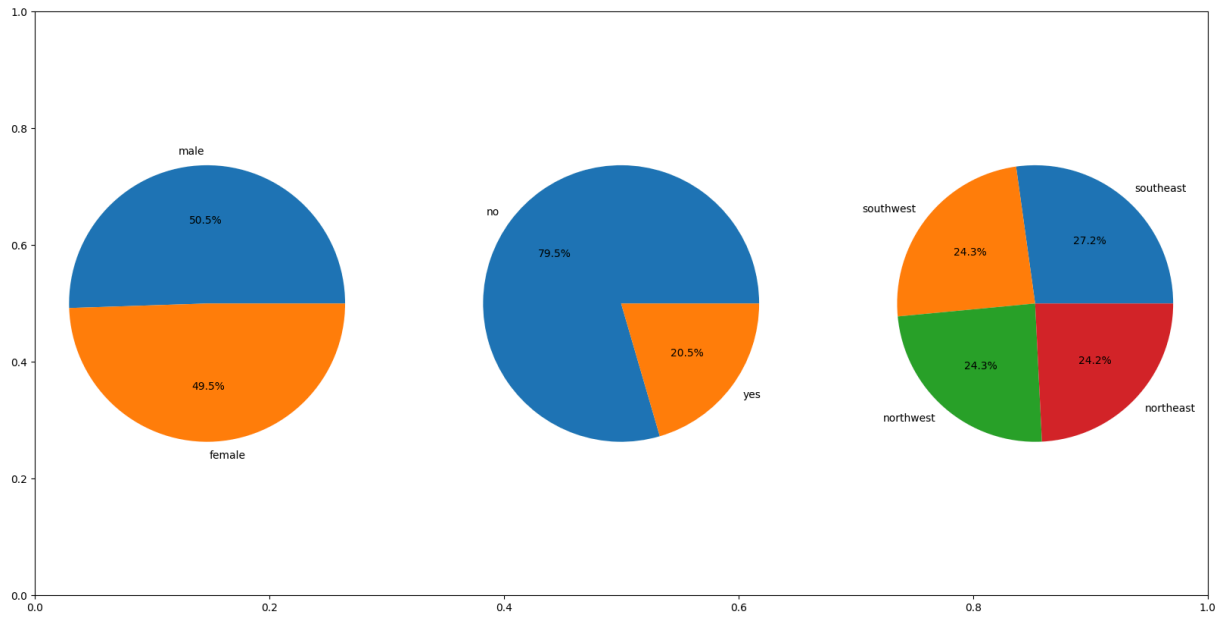
Out[4]:

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

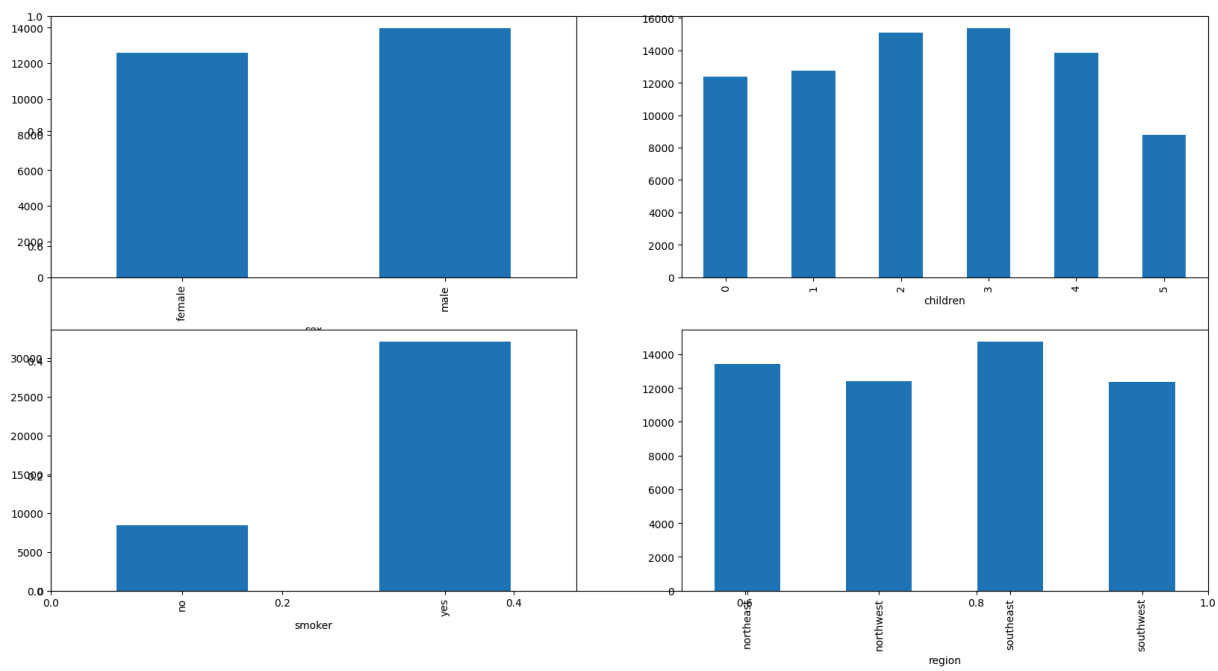
In [5]:
```
df.isnull().sum()
```

Out[5]:
```
age         0
sex         0
bmi         0
children    0
smoker      0
region      0
charges     0
dtype: int64
```

In [6]:
```
features = ['sex', 'smoker', 'region']

plt.subplots(figsize=(20, 10))
for i, col in enumerate(features):
    plt.subplot(1, 3, i + 1)

    x = df[col].value_counts()
    plt.pie(x.values,
            labels=x.index,
            autopct='%1.1f%%')

plt.show()
```

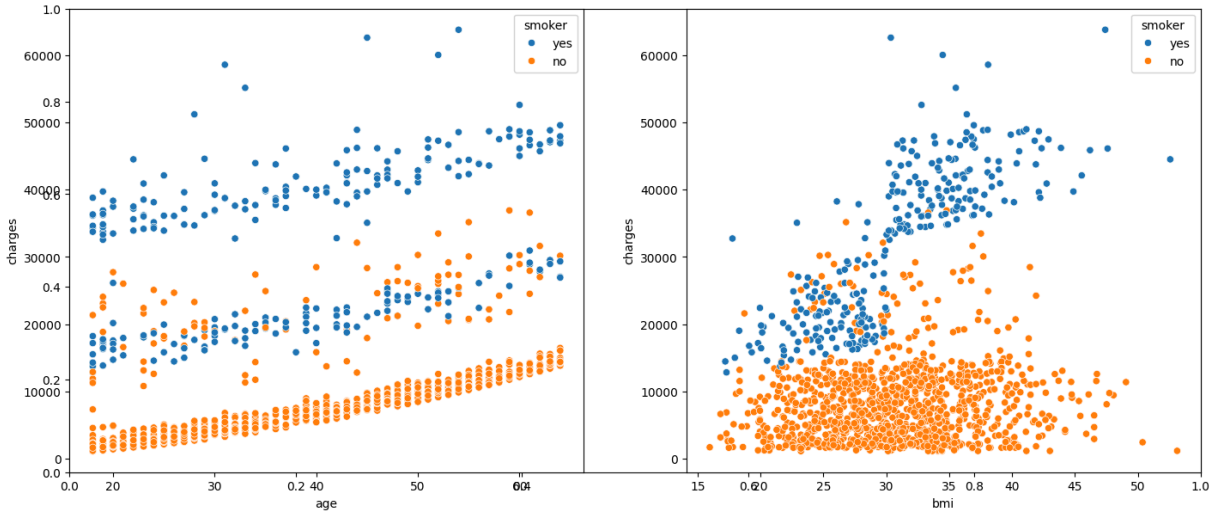


In [7]:
```
features = ['sex', 'children', 'smoker', 'region']

plt.subplots(figsize=(20, 10))
for i, col in enumerate(features):
    plt.subplot(2, 2, i + 1)
    df.groupby(col)['charges'].mean().astype(float).plot.bar()
plt.show()
```



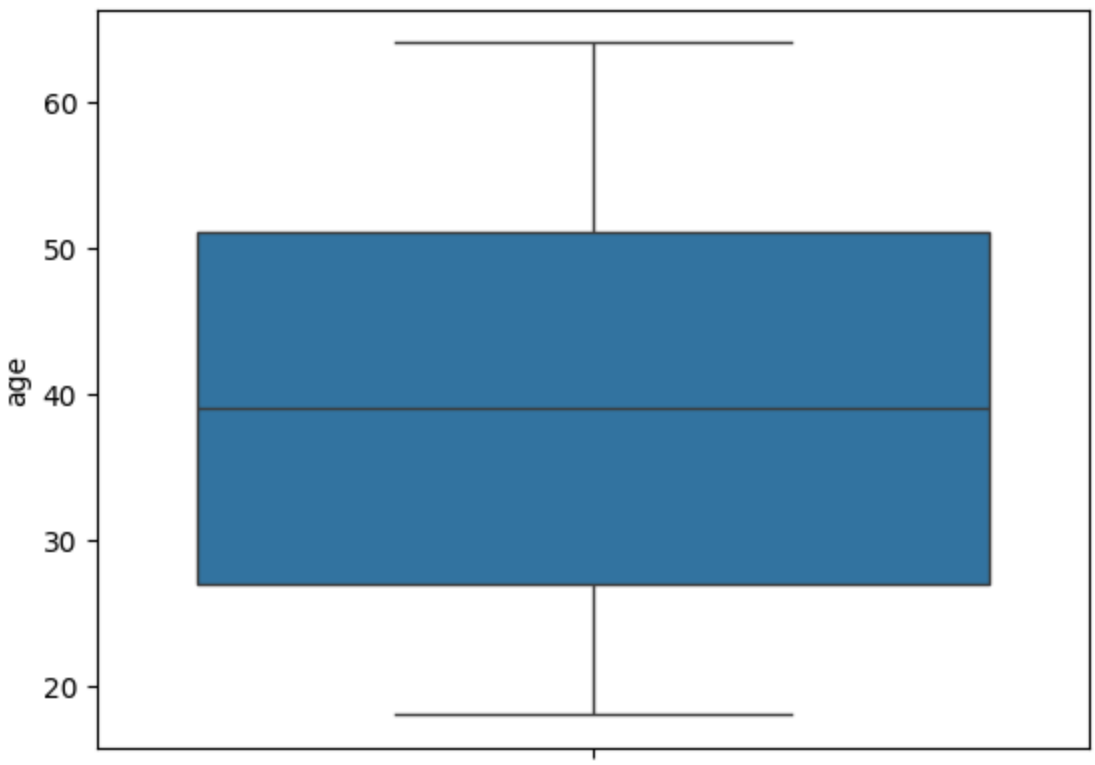
In [8]:
```
features = ['age', 'bmi']

plt.subplots(figsize=(17, 7))
for i, col in enumerate(features):
    plt.subplot(1, 2, i + 1)
    sns.scatterplot(data=df, x=col,
                    y='charges',
```

```
                    hue='smoker')
plt.show()
```



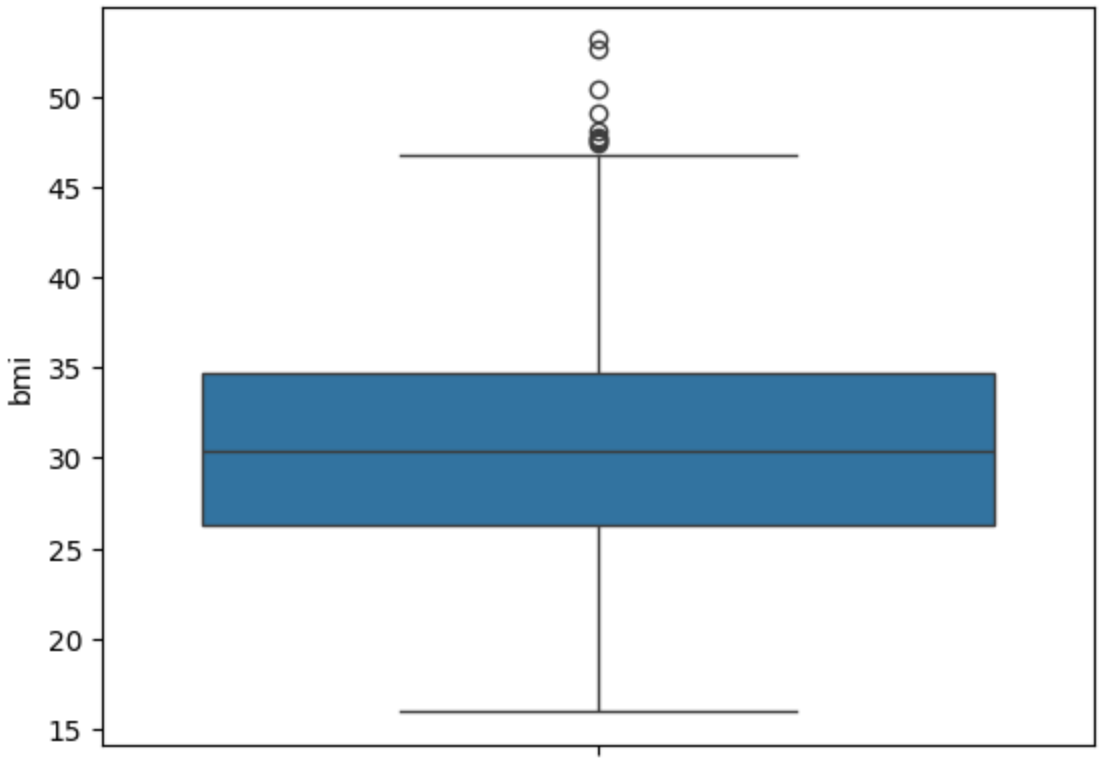
In [9]:
```
df.drop_duplicates(inplace=True)
sns.boxplot(df['age'])
```

Out[9]: <Axes: ylabel='age'>



In [10]:
```
sns.boxplot(df['bmi'])
```

Out[10]: <Axes: ylabel='bmi'>



In [11]:
```
Q1=df['bmi'].quantile(0.25)
Q2=df['bmi'].quantile(0.5)
Q3=df['bmi'].quantile(0.75)
iqr=Q3-Q1
lowlim=Q1-1.5*iqr
upplim=Q3+1.5*iqr
print(lowlim)
print(upplim)
```

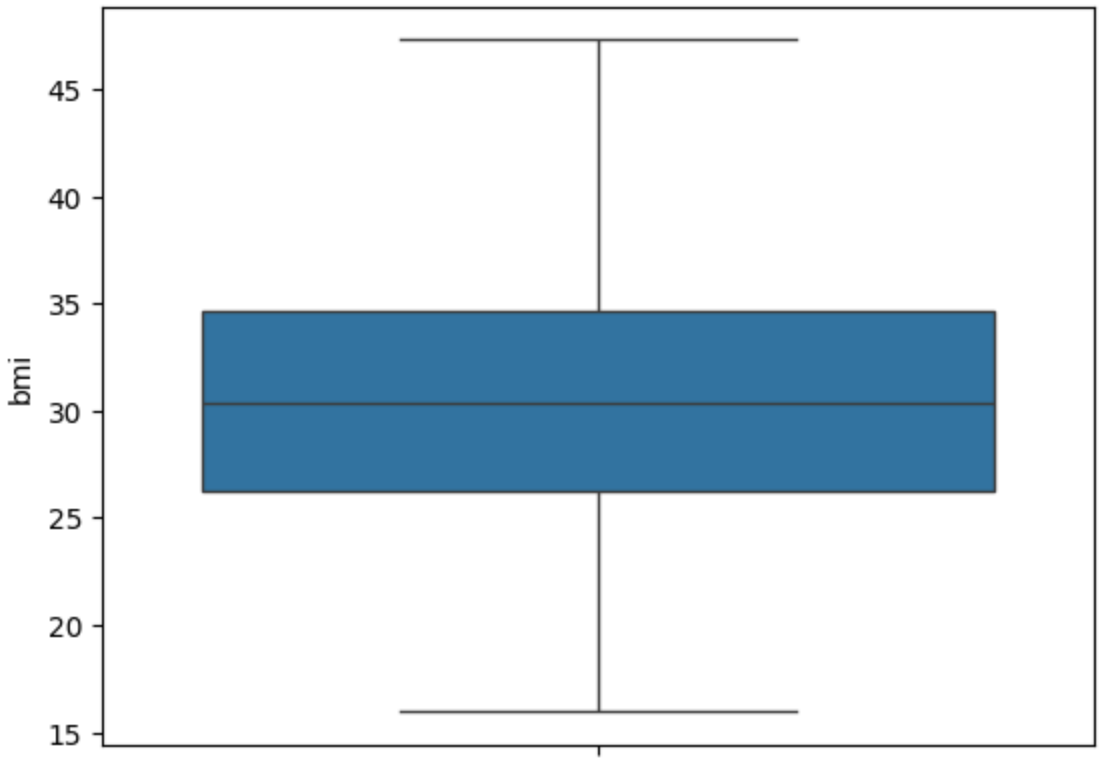
```
13.674999999999994
47.31500000000001
```

In [12]:
```
!pip install feature_engine
from feature_engine.outliers import ArbitraryOutlierCapper
arb=ArbitraryOutlierCapper(min_capping_dict={'bmi':13.6749},max_capping_dict={'bmi'
df[['bmi']]=arb.fit_transform(df[['bmi']])
sns.boxplot(df['bmi'])
```

```
Requirement already satisfied: feature_engine in c:\users\jotar\appdata\local\progra
ms\python\python313\lib\site-packages (1.8.3)
Requirement already satisfied: numpy>=1.18.2 in c:\users\jotar\appdata\local\program
s\python\python313\lib\site-packages (from feature_engine) (2.2.6)
Requirement already satisfied: pandas>=2.2.0 in c:\users\jotar\appdata\local\program
s\python\python313\lib\site-packages (from feature_engine) (2.2.3)
Requirement already satisfied: scikit-learn>=1.4.0 in c:\users\jotar\appdata\local\p
rograms\python\python313\lib\site-packages (from feature_engine) (1.6.1)
Requirement already satisfied: scipy>=1.4.1 in c:\users\jotar\appdata\local\programs
\python\python313\lib\site-packages (from feature_engine) (1.15.3)
Requirement already satisfied: statsmodels>=0.11.1 in c:\users\jotar\appdata\local\p
rograms\python\python313\lib\site-packages (from feature_engine) (0.14.4)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\jotar\appdata\loca
l\programs\python\python313\lib\site-packages (from pandas>=2.2.0->feature_engine)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\jotar\appdata\local\programs
\python\python313\lib\site-packages (from pandas>=2.2.0->feature_engine) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\jotar\appdata\local\progra
ms\python\python313\lib\site-packages (from pandas>=2.2.0->feature_engine) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\jotar\appdata\local\programs\pyt
hon\python313\lib\site-packages (from python-dateutil>=2.8.2->pandas>=2.2.0->feature
_engine) (1.16.0)
Requirement already satisfied: joblib>=1.2.0 in c:\users\jotar\appdata\local\program
s\python\python313\lib\site-packages (from scikit-learn>=1.4.0->feature_engine) (1.
5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\jotar\appdata\local
\programs\python\python313\lib\site-packages (from scikit-learn>=1.4.0->feature_engi
ne) (3.6.0)
Requirement already satisfied: patsy>=0.5.6 in c:\users\jotar\appdata\local\programs
\python\python313\lib\site-packages (from statsmodels>=0.11.1->feature_engine) (1.0.
1)
Requirement already satisfied: packaging>=21.3 in c:\users\jotar\appdata\local\progr
ams\python\python313\lib\site-packages (from statsmodels>=0.11.1->feature_engine) (2
4.1)
```

Out[12]:

```
<Axes: ylabel='bmi'>
```



In [13]:
```
print(df['bmi'].skew())
print(df['age'].skew())
```

```
0.23289153320569975
0.054780773126998195
```

In [14]:
```
df['sex']=df['sex'].map({'male':0,'female':1})
df['smoker']=df['smoker'].map({'yes':1,'no':0})
df['region']=df['region'].map({'northwest':0, 'northeast':1,'southeast':2,'southwes
```

In [15]:
```
df
```

Out[15]:

	age	sex	bmi	children	smoker	region	charges
0	19	1	27.900	0	1	3	16884.92400
1	18	0	33.770	1	0	2	1725.55230
2	28	0	33.000	3	0	2	4449.46200
3	33	0	22.705	0	0	0	21984.47061
4	32	0	28.880	0	0	0	3866.85520
...	...	...	...	...	...	...	...
1333	50	0	30.970	3	0	0	10600.54830
1334	18	1	31.920	0	0	1	2205.98080
1335	18	1	36.850	0	0	2	1629.83350
1336	21	1	25.800	0	0	3	2007.94500
1337	61	1	29.070	0	1	0	29141.36030

1337 rows × 7 columns

In [16]:

```
print(df.head())
df.corr()
```

```
   age  sex     bmi  children  smoker  region      charges
0   19    1  27.900         0       1       3  16884.92400
1   18    0  33.770         1       0       2   1725.55230
2   28    0  33.000         3       0       2   4449.46200
3   33    0  22.705         0       0       0  21984.47061
4   32    0  28.880         0       0       0   3866.85520
```

Out[16]:

	age	sex	bmi	children	smoker	region	charges
age	1.000000	0.019814	0.111998	0.041536	-0.025587	0.001771	0.298308
sex	0.019814	1.000000	-0.044831	-0.017848	-0.076596	-0.008998	-0.058044
bmi	0.111998	-0.044831	1.000000	0.013692	0.003151	0.156937	0.199063
children	0.041536	-0.017848	0.013692	1.000000	0.007331	-0.002842	0.067389
smoker	-0.025587	-0.076596	0.003151	0.007331	1.000000	0.012736	0.787234
region	0.001771	-0.008998	0.156937	-0.002842	0.012736	1.000000	0.010767
charges	0.298308	-0.058044	0.199063	0.067389	0.787234	0.010767	1.000000

In [17]:

```
! pip install xgboost
```

```
Requirement already satisfied: xgboost in c:\users\jotar\appdata\local\programs\pyth
on\python313\lib\site-packages (3.0.2)
Requirement already satisfied: numpy in c:\users\jotar\appdata\local\programs\python
\python313\lib\site-packages (from xgboost) (2.2.6)
Requirement already satisfied: scipy in c:\users\jotar\appdata\local\programs\python
\python313\lib\site-packages (from xgboost) (1.15.3)
```

In [18]:

```
X=df.drop(['charges'],axis=1)
Y=df[['charges']]
from sklearn.linear_model import LinearRegression,Lasso
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
l1=[]
l2=[]
l3=[]
cvs=0
for i in range(40,50):
 xtrain,xtest,ytrain,ytest=train_test_split(X,Y,test_size=0.2,random_state=i)
 lrmodel=LinearRegression()
 lrmodel.fit(xtrain,ytrain)
 l1.append(lrmodel.score(xtrain,ytrain))
 l2.append(lrmodel.score(xtest,ytest))
 cvs=(cross_val_score(lrmodel,X,Y,cv=5,)).mean()
 l3.append(cvs)
 df1=pd.DataFrame({'train acc':l1,'test acc':l2,'cvs':l3})
df1
```

Out[18]:

	train acc	test acc	cvs
0	0.741659	0.778409	0.74707
1	0.756401	0.706267	0.74707
2	0.729542	0.806239	0.74707
3	0.754260	0.732791	0.74707
4	0.742966	0.779591	0.74707
5	0.753281	0.731769	0.74707
6	0.741261	0.776456	0.74707
7	0.731940	0.796173	0.74707
8	0.751915	0.741742	0.74707
9	0.756348	0.722565	0.74707

In [19]:

```
xtrain,xtest,ytrain,ytest=train_test_split(X,Y,test_size=0.2,random_state=42)
lrmodel=LinearRegression()
lrmodel.fit(xtrain,ytrain)
print("linear regression: ")
```

```
print(lrmodel.score(xtrain,ytrain))
print(lrmodel.score(xtest,ytest))
print(cross_val_score(lrmodel,X,Y,cv=5,).mean())
```

```
linear regression:
0.7295415541376447
0.8062391115570589
0.7470697972809902
```

In [20]:
```
from sklearn.metrics import r2_score
svrmodel=SVR()
svrmodel.fit(xtrain,ytrain)
ypredtrain1=svrmodel.predict(xtrain)
ypredtest1=svrmodel.predict(xtest)
print("SVR")
print(r2_score(ytrain,ypredtrain1))
print(r2_score(ytest,ypredtest1))
print(cross_val_score(svrmodel,X,Y,cv=5,).mean())
```

```
SVR
-0.10151474302536445
-0.1344454720199666
-0.10374591327267262
```

In [21]:
```
rfmodel=RandomForestRegressor(random_state=42)
rfmodel.fit(xtrain,ytrain)
ypredtrain2=rfmodel.predict(xtrain)
ypredtest2=rfmodel.predict(xtest)
print("random forest regressor")
print(r2_score(ytrain,ypredtrain2))
print(r2_score(ytest,ypredtest2))
print(cross_val_score(rfmodel,X,Y,cv=5,).mean())
from sklearn.model_selection import GridSearchCV
estimator=RandomForestRegressor(random_state=42)
param_grid={'n_estimators':[10,40,50,98,100,120,150]}
grid=GridSearchCV(estimator,param_grid,scoring="r2",cv=5)
grid.fit(xtrain,ytrain)
print("Hyperparametertuning:")
print(grid.best_params_)
rfmodel=RandomForestRegressor(random_state=42,n_estimators=120)
rfmodel.fit(xtrain,ytrain)
ypredtrain2=rfmodel.predict(xtrain)
ypredtest2=rfmodel.predict(xtest)
print(r2_score(ytrain,ypredtrain2))
print(r2_score(ytest,ypredtest2))
print(cross_val_score(rfmodel,X,Y,cv=5,).mean())
```

```
random forest regressor
0.9738163260247533
0.8819423353068565
0.8363637309718952
Hyperparametertuning:
{'n_estimators': 120}
0.9746383984429655
0.8822009842175969
0.8367438097052858
```

In [22]:
```
gbmodel=GradientBoostingRegressor()
gbmodel.fit(xtrain,ytrain)
ypredtrain3=gbmodel.predict(xtrain)
ypredtest3=gbmodel.predict(xtest)
print("gradient boost regressor")
print(r2_score(ytrain,ypredtrain3))
print(r2_score(ytest,ypredtest3))
print(cross_val_score(gbmodel,X,Y,cv=5,).mean())
from sklearn.model_selection import GridSearchCV
estimator=GradientBoostingRegressor()
param_grid={'n_estimators':[10,15,19,20,21,50],'learning_rate':[0.1,0.19,0.2,0.21,0
grid=GridSearchCV(estimator,param_grid,scoring="r2",cv=5)
grid.fit(xtrain,ytrain)
print("Hyperparametertuning")
print(grid.best_params_)
gbmodel=GradientBoostingRegressor(n_estimators=19,learning_rate=0.2)
gbmodel.fit(xtrain,ytrain)
ypredtrain3=gbmodel.predict(xtrain)
ypredtest3=gbmodel.predict(xtest)
print(r2_score(ytrain,ypredtrain3))
print(r2_score(ytest,ypredtest3))
print(cross_val_score(gbmodel,X,Y,cv=5,).mean())
```

```
gradient boost regressor
0.8931345821166041
0.9042488247824163
0.854834812494289
Hyperparametertuning
{'learning_rate': 0.2, 'n_estimators': 19}
0.8682397447116926
0.9017109716082661
0.86051471152677
```

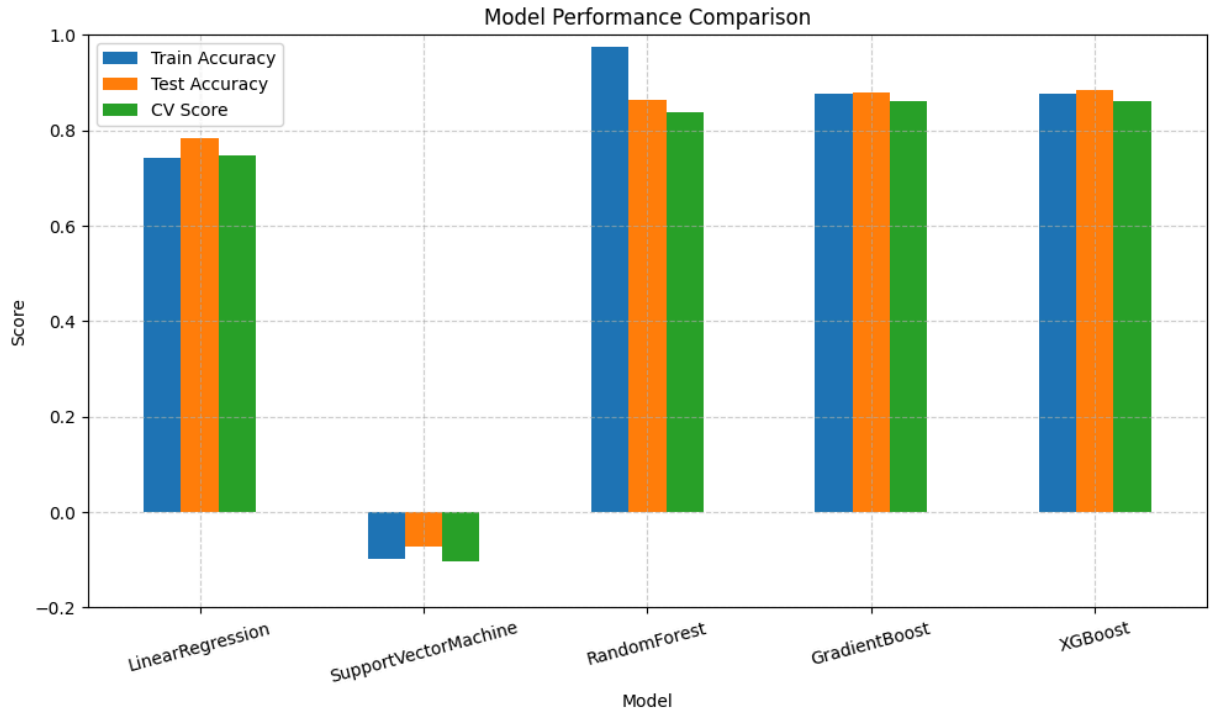
In [23]:
```
xgmodel=XGBRegressor()
xgmodel.fit(xtrain,ytrain)
ypredtrain4=xgmodel.predict(xtrain)
ypredtest4=xgmodel.predict(xtest)
print("XGB regressor")
print(r2_score(ytrain,ypredtrain4))
print(r2_score(ytest,ypredtest4))
print(cross_val_score(xgmodel,X,Y,cv=5,).mean())
from sklearn.model_selection import GridSearchCV
estimator=XGBRegressor()
param_grid={'n_estimators':[10,15,20,40,50],'max_depth':[3,4,5],'gamma':[0,0.15,0.3
grid=GridSearchCV(estimator,param_grid,scoring="r2",cv=5)
grid.fit(xtrain,ytrain)
print("")
print(grid.best_params_)
xgmodel=XGBRegressor(n_estimators=15,max_depth=3,gamma=0)
xgmodel.fit(xtrain,ytrain)
ypredtrain4=xgmodel.predict(xtrain)
ypredtest4=xgmodel.predict(xtest)
print(r2_score(ytrain,ypredtrain4))
print(r2_score(ytest,ypredtest4))
print(cross_val_score(xgmodel,X,Y,cv=5,).mean())
```

```
XGB regressor
0.9954123497009277
0.8548938035964966
0.8081253051757813

{'gamma': 0, 'max_depth': 3, 'n_estimators': 10}
0.869317352771759
0.9022461175918579
0.8607115387916565
```

In [24]:

```python
import pandas as pd
import matplotlib.pyplot as plt

# Performance data
data = {
    'Model': ['LinearRegression', 'SupportVectorMachine', 'RandomForest', 'Gradient
    'Train Accuracy': [0.7414104501076269, -0.09774269027100302, 0.9742433796570987
    'Test Accuracy': [0.7826938736089379, -0.07230814309874001,0.8634686176821831 ,
    'CV Score': [0.7470910325728581, -0.1036292241111501, 0.8367985697519954, 0.860
}

# Create DataFrame
comparison_df = pd.DataFrame(data)

# Display the DataFrame
print(comparison_df)

# Optional: Visualize comparison using bar plots
comparison_df.set_index('Model').plot(kind='bar', figsize=(10, 6), ylim=(-0.2, 1.0)
plt.title('Model Performance Comparison')
plt.ylabel('Score')
plt.xticks(rotation=15)
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

```
                  Model  Train Accuracy  Test Accuracy  CV Score
0      LinearRegression        0.741410       0.782694  0.747091
1  SupportVectorMachine       -0.097743      -0.072308 -0.103629
2          RandomForest        0.974243       0.863469  0.836799
3         GradientBoost        0.876993       0.879012  0.860724
4               XGBoost        0.877876       0.885432  0.861775
```



In [25]:
```
feats=pd.DataFrame(data=grid.best_estimator_.feature_importances_,index=X.columns,c
feats
```

Out[25]:

	Importance
age	0.038633
sex	0.000000
bmi	0.133449
children	0.011073
smoker	0.809626
region	0.007219

In [26]:
```
important_features=feats[feats['Importance']>0.01]
important_features
```

Out[26]:

	Importance
age	0.038633
bmi	0.133449
children	0.011073
smoker	0.809626

In [27]:
```
print(df.columns)
```

```
Index(['age', 'sex', 'bmi', 'children', 'smoker', 'region', 'charges'], dtype='objec
t')
```

In [28]:
```
# final model
df.drop(['sex', 'region'], axis=1, inplace=True)
Xf=df.drop(df[['charges']],axis=1)
X=df.drop(df[['charges']],axis=1)
xtrain,xtest,ytrain,ytest=train_test_split(Xf,Y,test_size=0.2,random_state=42)
finalmodel=XGBRegressor(n_estimators=15,max_depth=3,gamma=0)
finalmodel.fit(xtrain,ytrain)
ypredtrain4=finalmodel.predict(xtrain)
ypredtest4=finalmodel.predict(xtest)
print("Final Model:")
print("train accuracy: ")
print(r2_score(ytrain,ypredtrain4))
print("test accuracy: ")
print(r2_score(ytest,ypredtest4))
print("cv score: " )
print(cross_val_score(finalmodel,X,Y,cv=5,).mean())
```

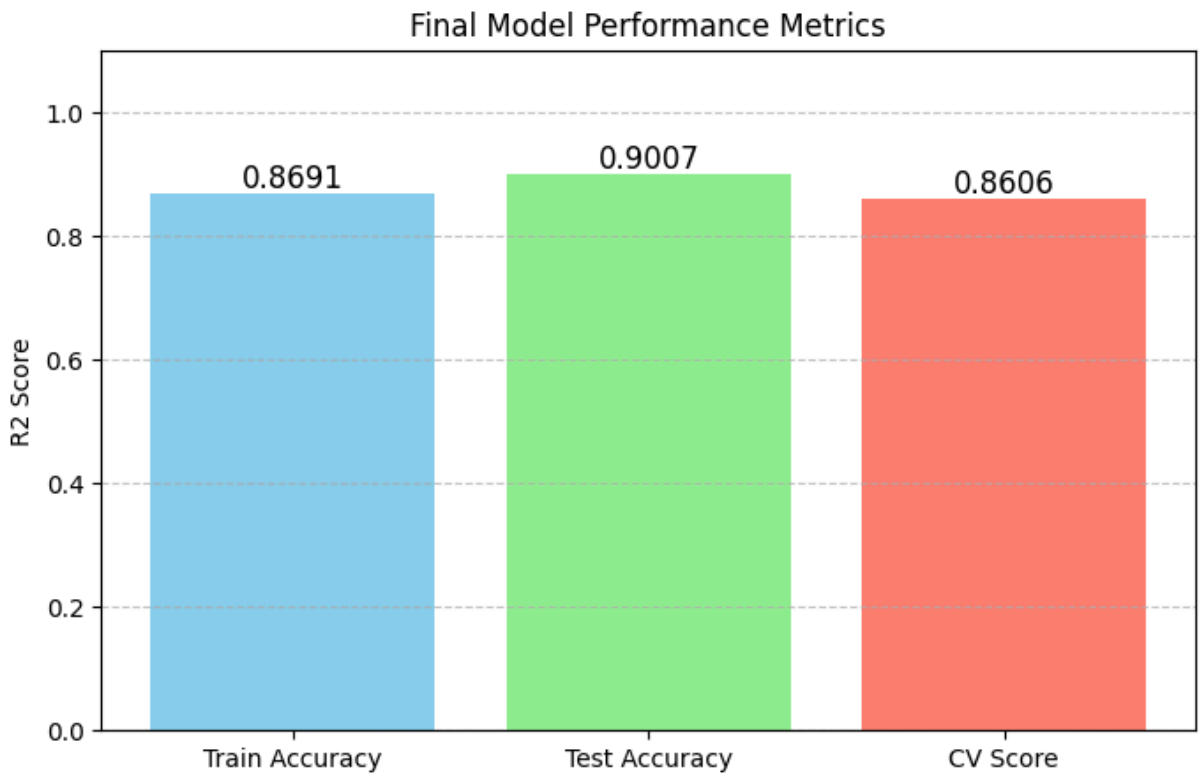
```
Final Model:
train accuracy:
0.8691051602363586
test accuracy:
0.9007425308227539
cv score:
0.8606266975402832
```

In [29]:
```
print(Xf.select_dtypes(include='object').columns)
```

```
Index([], dtype='object')
```

In [30]:
```
import matplotlib.pyplot as plt
from sklearn.metrics import r2_score
from sklearn.model_selection import cross_val_score


train_acc = r2_score(ytrain, ypredtrain4)
test_acc = r2_score(ytest, ypredtest4)
cv_score = cross_val_score(finalmodel, X, Y, cv=5).mean()

# Prepare data for plotting
metrics = ['Train Accuracy', 'Test Accuracy', 'CV Score']
scores = [train_acc, test_acc, cv_score]

# Plotting
plt.figure(figsize=(8,5))
bars = plt.bar(metrics, scores, color=['skyblue', 'lightgreen', 'salmon'])

# Add score labels on top of bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.01, round(yval, 4), ha='cent

plt.ylim(0, 1.1)
plt.title('Final Model Performance Metrics')
plt.ylabel('R2 Score')
```

```
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



In [ ]:

In [ ]:

In [ ]: